

AIL 722: Assignment 4 - Reinforcement Learning

Aditya Saxena
Entry Number: 2022AM11218

November 2025

1 DQN Overestimation (LunarLander-v3)

1.1 Implementation Details

We implemented both DQN and Double DQN agents using the LunarLander-v3 environment. Both agents use the same network architecture and hyperparameters.

Network Architecture

- Input Layer: Dimension equals observation space.
- Two hidden layers: 512 units each, ReLU activation.
- Output Layer: Linear layer mapping to number of actions.

Hyperparameters

Parameter	Value
Learning Rate	3×10^{-4}
Discount Factor γ	0.99
Batch Size	64
Replay Buffer Size	100,000
Target Update	Every 100 steps
Exploration ϵ	$1.0 \rightarrow 0.01$ (decay $\tau = 0.995$)

Table 1: Hyperparameters used for both DQN and Double DQN agents.

1.2 Training Rewards

The agents were trained for 2500 steps. Figure 1 shows the episode returns.

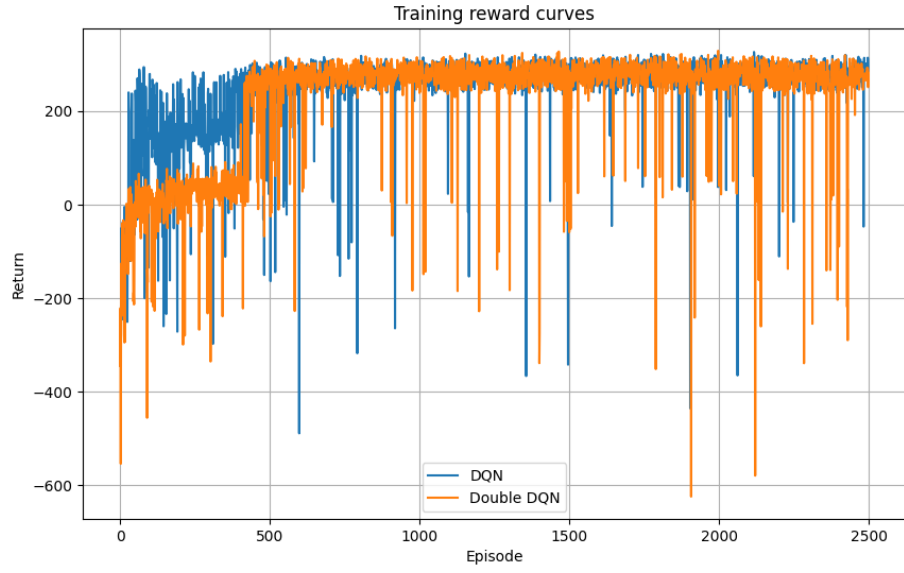


Figure 1: Training rewards (episode return) for DQN vs Double DQN.

1.3 Evaluation Results

Agent	Mean Return	Std. Dev.
DQN	274.7656	38.8676
Double DQN	274.6611	26.5649

Table 2: Evaluation statistics over 100 episodes.

1.4 Per-action Q-value Comparison

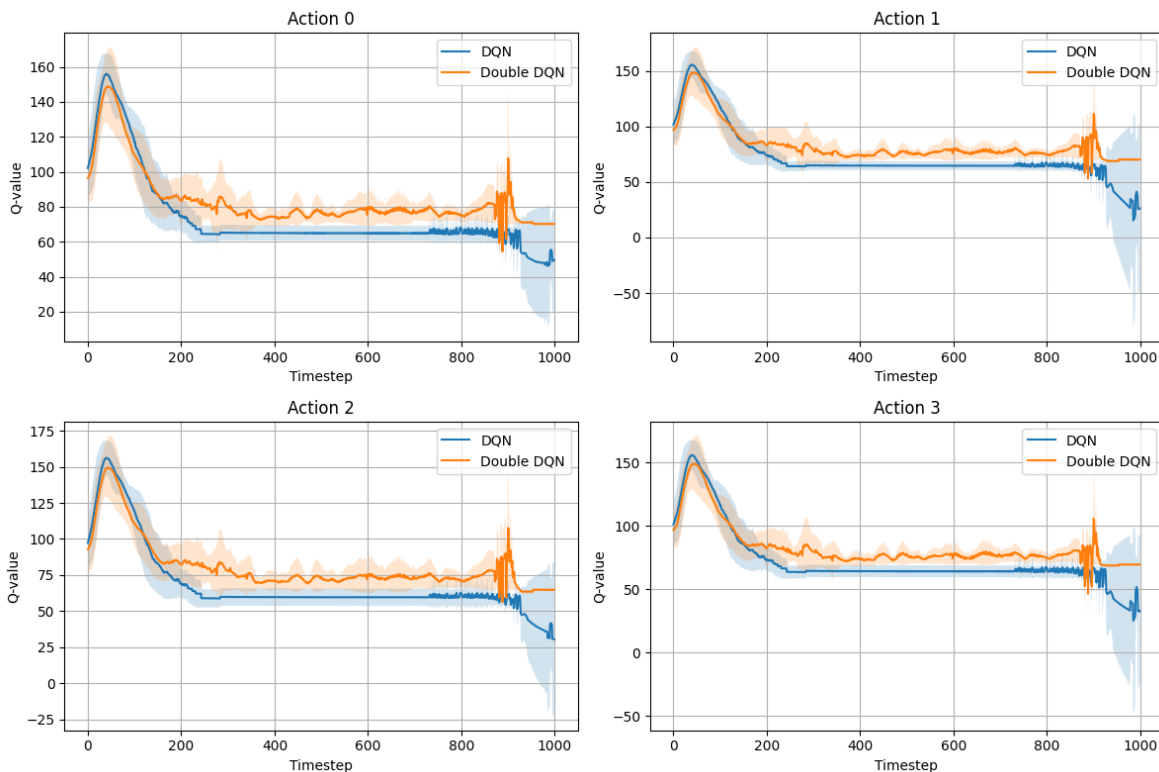


Figure 2: Comparison of predicted Q-values per action for DQN and Double DQN.

1.5 Observations on Overestimation

- Standard DQN consistently predicts higher Q-values than Double DQN.
- Confirms presence of overestimation bias due to max operator.
- Double DQN reduces this by decoupling action selection from evaluation.

2 Gradient Variance Analysis of REINFORCE Baselines

2.1 Implementation Details

We implemented REINFORCE on InvertedPendulum-v4 with four baselines:

1. No Baseline
2. Average Reward
3. Reward-to-Go
4. Learned Value Function $V(s)$

Hyperparameters

Parameter	Value
Policy Learning Rate	1×10^{-3}
Discount Factor γ	0.99
Hidden Layer Size	128
Target Reward Range	450–500
Max Episodes	100000

Table 3: Hyperparameters for REINFORCE baselines.

2.2 Variance Experiment

500 trajectories were collected per baseline. For sample sizes $N \in \{20, 30, \dots, 100\}$:

- Sample N trajectories
- Compute gradient norm
- Repeat 10 times

2.3 Results

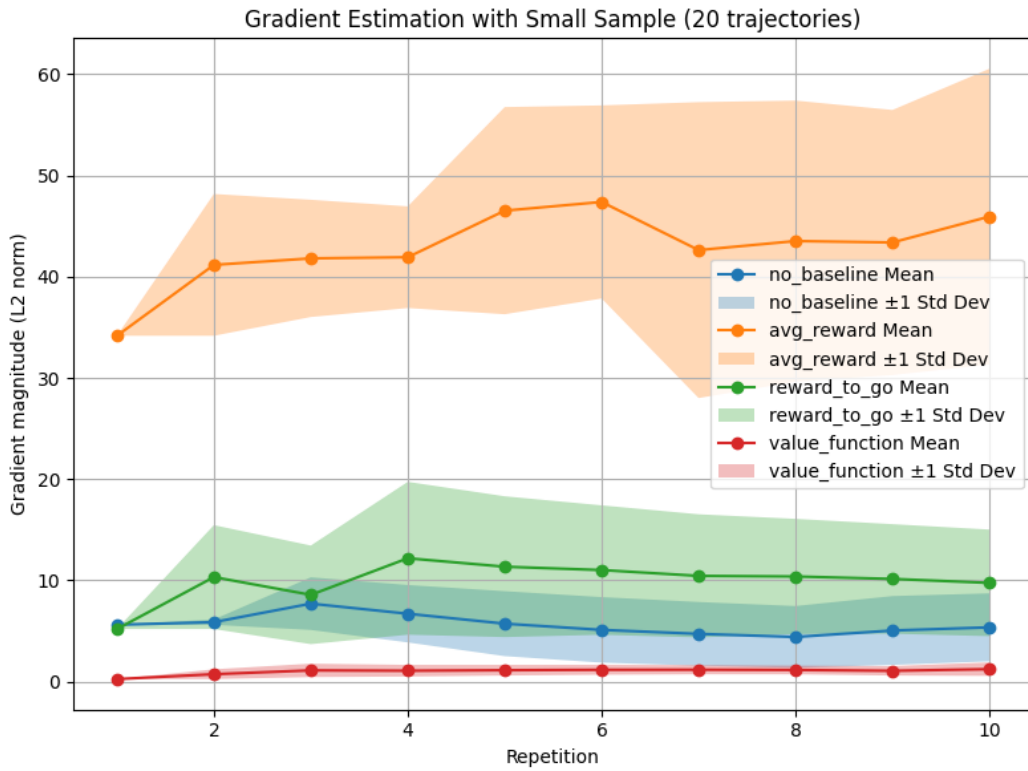


Figure 3: Gradient estimate magnitude vs. sample size for different baselines.

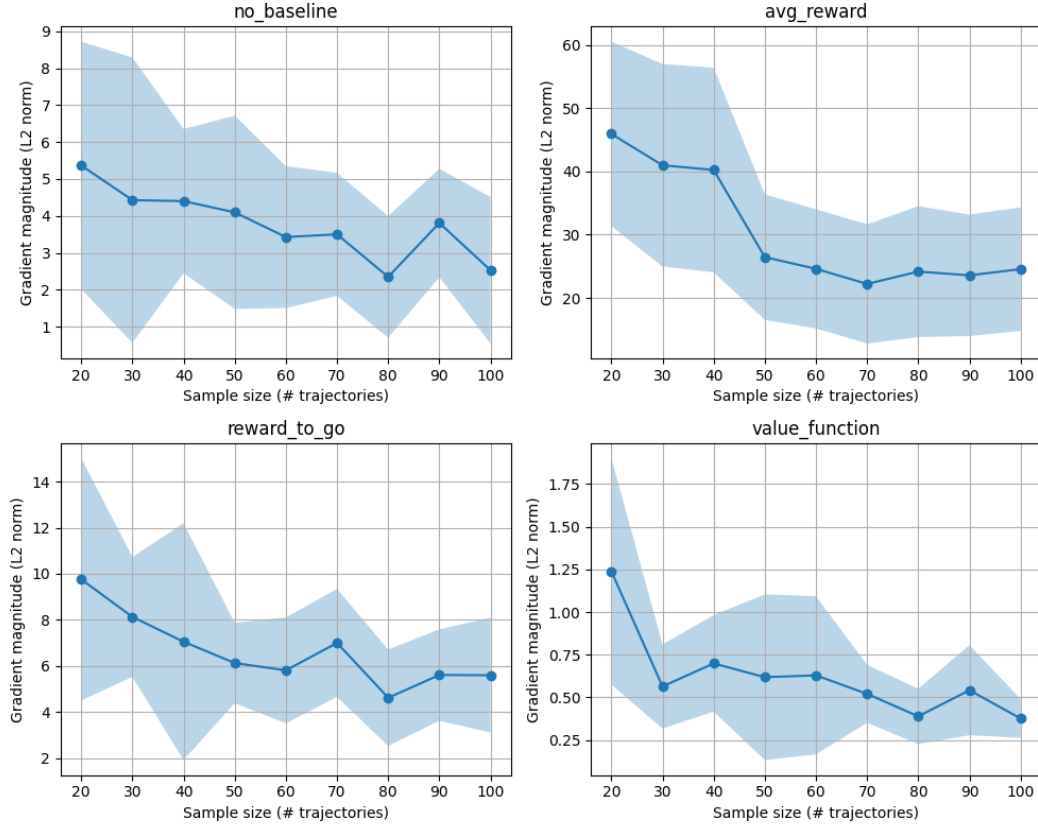


Figure 4: Gradient estimate and variance magnitude for different baselines for samall sample(20) multiple(10)times

Observations()

- Sample size does not have much effect on gradient magnitudes and variances
- Since the algorithm without baseline does not converge no meaning analysis could be done from it
- Average reward baseline produces high magnitude and variance.
- Reward to-go baseline produces lower gradients with less variance
- Value function baseline produces smallest gradients but tighter bands at $N \approx 100$.

3 Stable Baselines (A2C vs PPO)

3.1 Hyperparameter Tuning

Table 4: Hyperparameters for PPO and A2C on InvertedPendulum-v4

Hyperparameter	PPO	A2C
Algorithm	PPO	A2C
Total Timesteps	100,000	100,000
Log Interval	500	500
Eval Frequency	5000	5000
Save Frequency	50,000	50,000
Run Name	Run	Run
Learning Rate	3×10^{-4}	7×10^{-4}
n_{steps}	2048	5
Batch Size	64	–
n_{epochs}	10	–
γ	0.95	0.95
GAE λ	0.95	0.95
Clip Range	0.2	–
Entropy Coefficient	0.0	0.05
Value Function Coef	0.5	0.5
Max Grad Norm	0.5	0.5
Use RMSProp	–	True

Table 5: Hyperparameters for PPO and A2C on HalfCheetah-v4

Hyperparameter	PPO	A2C
Algorithm	PPO	A2C
Total Timesteps	1,000,000	1,250,000
Log Interval	500	500
Eval Frequency	5000	5000
Save Frequency	50,000	50,000
Run Name	Run	Run
Learning Rate	3×10^{-4}	5×10^{-4}
n_{steps}	2048	20
Batch Size	64	–
n_{epochs}	10	–
γ	0.99	0.99
GAE λ	0.95	0.95
Clip Range	0.2	–
Entropy Coefficient	0.05	0.05
Value Function Coef	0.5	0.5
Max Grad Norm	0.5	1.0
Use RMSProp	–	True

Table 6: Hyperparameters for PPO and A2C on Hopper-v4

Hyperparameter	PPO	A2C
Algorithm	PPO	A2C
Total Timesteps	1,000,000	1,250,000
Log Interval	500	500
Eval Frequency	5000	5000
Save Frequency	50,000	50,000
Run Name	Run	Run
Learning Rate	3×10^{-4}	7×10^{-4}
n_{steps}	2048	20
Batch Size	64	—
n_{epochs}	10	—
γ	0.99	0.99
GAE λ	0.95	0.95
Clip Range	0.2	—
Entropy Coefficient	0.0	0.01
Value Function Coef	0.5	0.5
Max Grad Norm	0.5	1.0
Use RMSProp	—	True

3.2 Tensorboard Plots

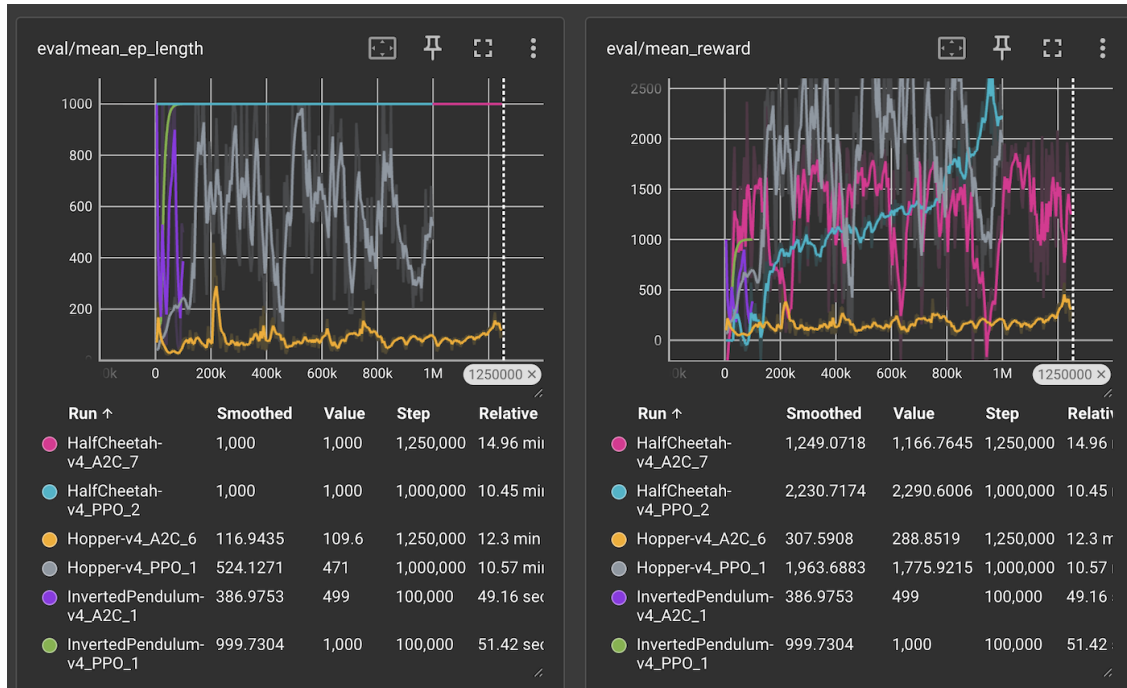


Figure 5: Eval Mean reward and Episode Mean Length



Figure 6: Rollout Mean reward and Episode Mean Length

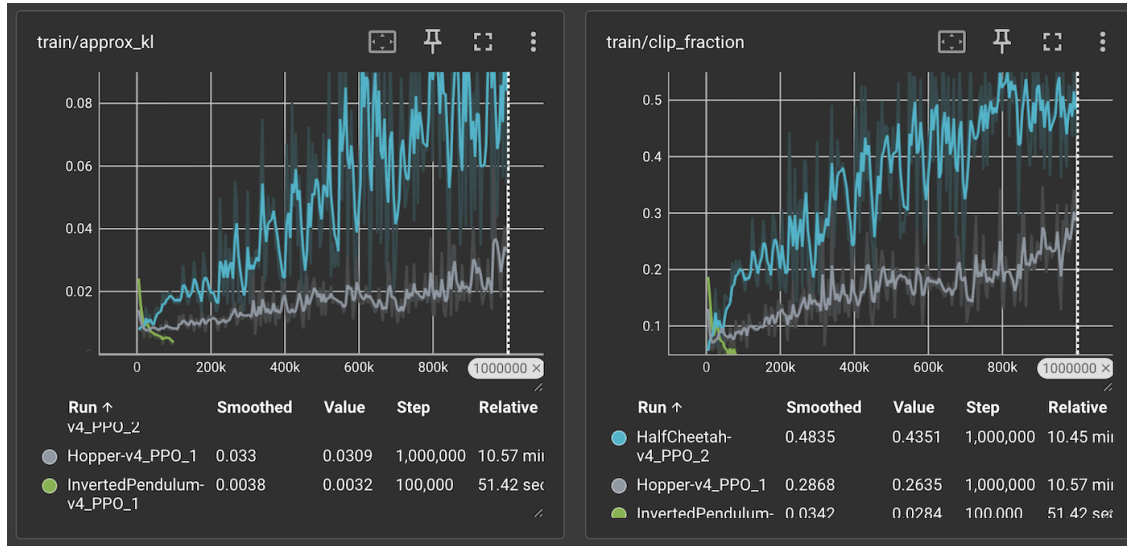


Figure 7: Training KL divergence and clip fraction

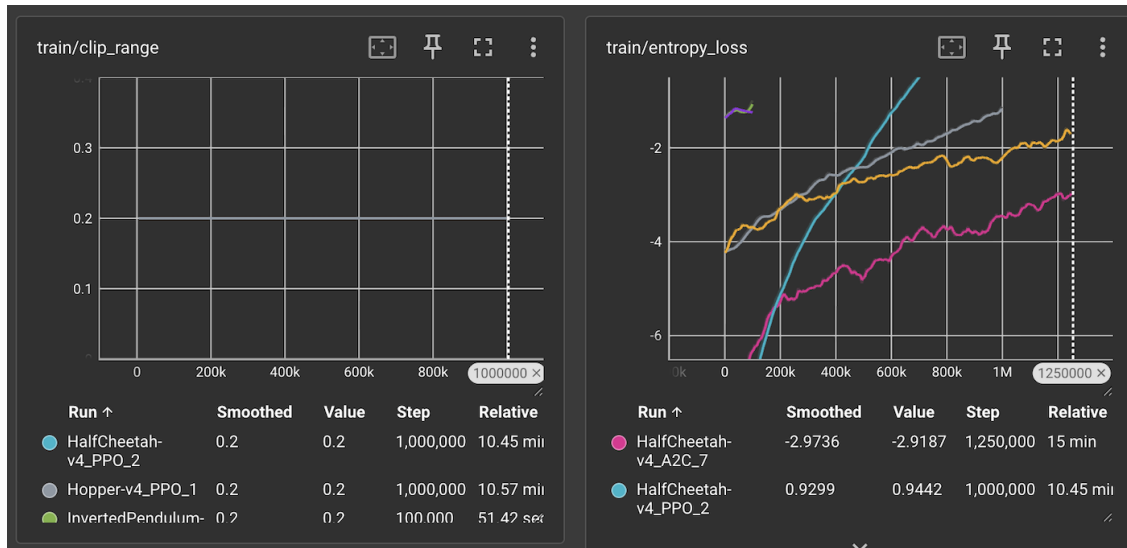


Figure 8: Training clip range and Entropy Loss



Figure 9: Training explained variance and learning rate

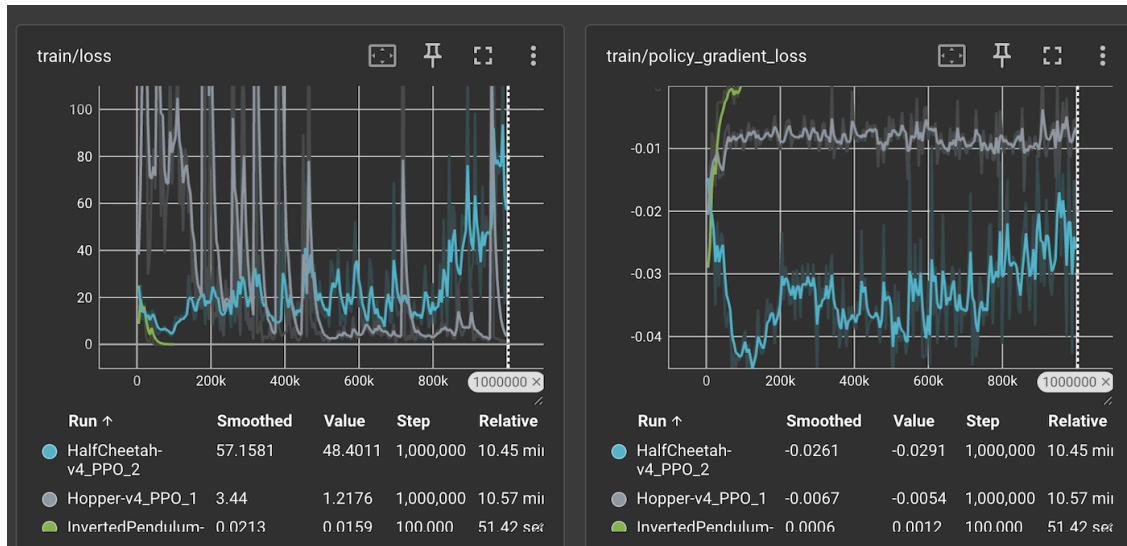


Figure 10: Training loss and policy gradient loss

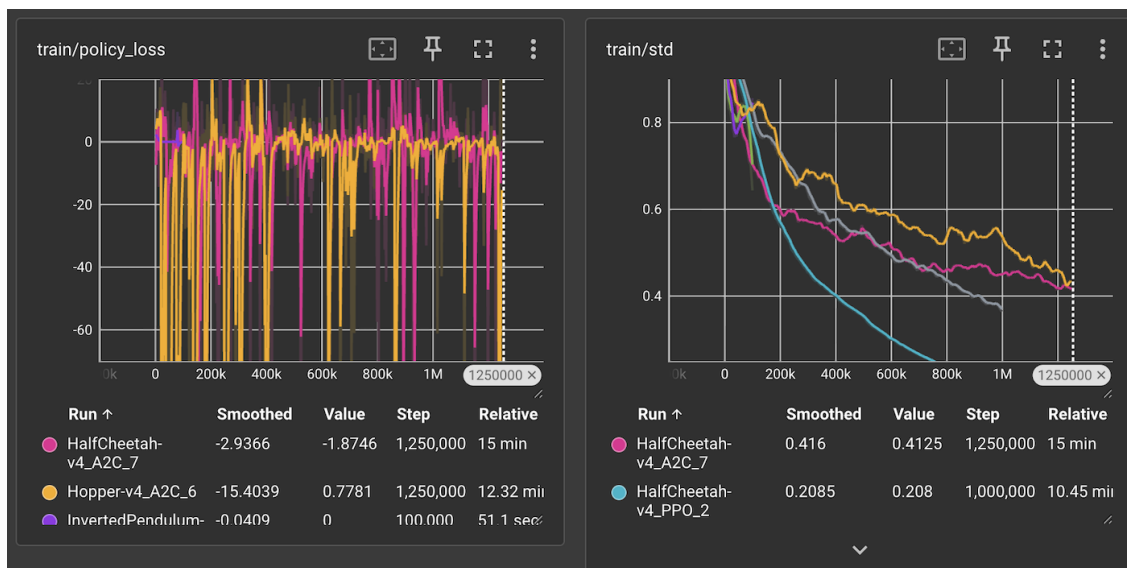


Figure 11: Training policy loss and std dev

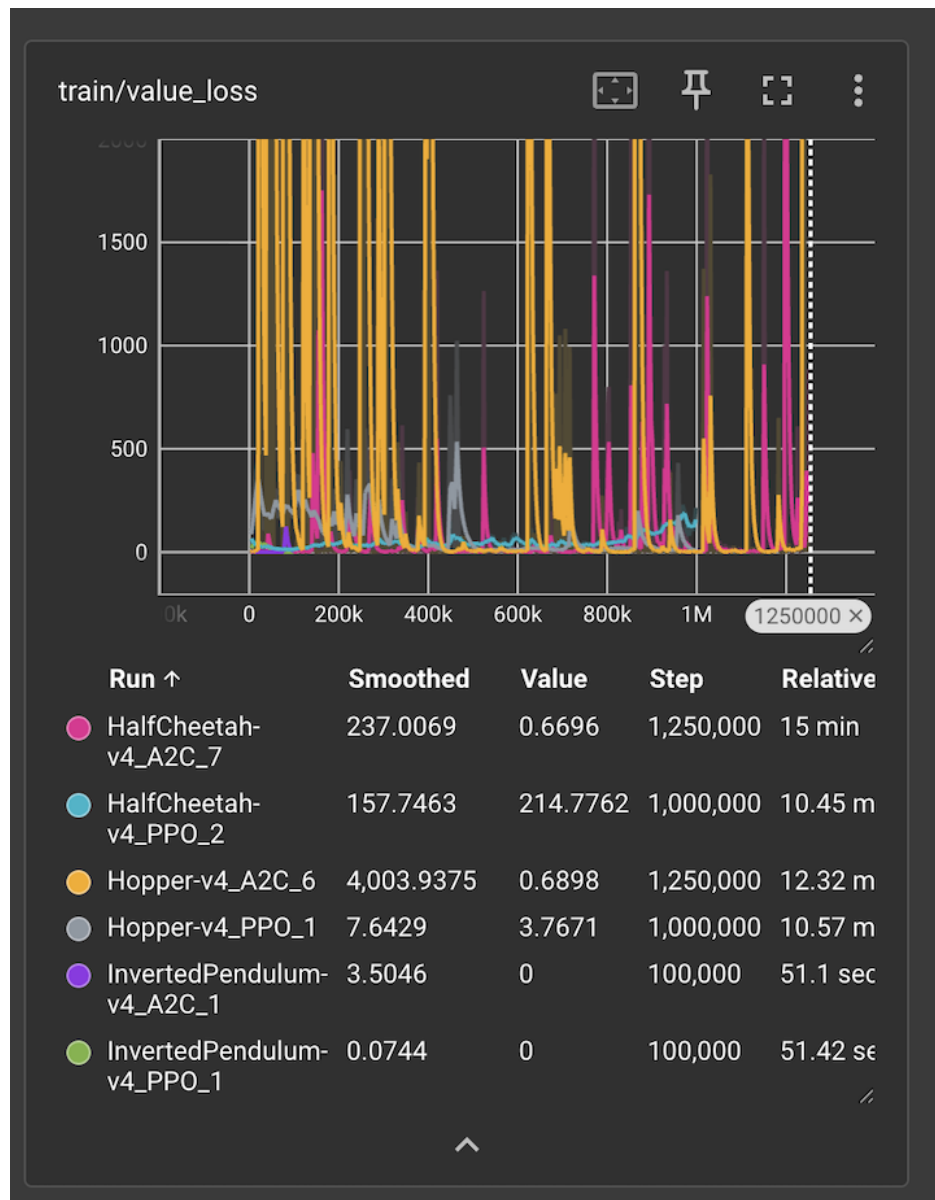


Figure 12: Training value loss

	HalfCheetah-v4_A2C_7	
	HalfCheetah-v4_PPO_2	
	Hopper-v4_A2C_6	
	Hopper-v4_PPO_1	
	InvertedPendulum-v4_A2C_1	
	InvertedPendulum-v4_PPO_1	

Figure 13: Legend

3.3 Findings

- PPO is more stable in Hopper and HalfCheetah.
- Entropy coefficient crucial for exploration.
- PPO shows better sample efficiency overall.

4 Policy Gradients vs DQN (LunarLander-v3)

4.1 Implementation Details

Both Actor-Critic and DQN use 2 hidden layers of 128 units.

4.2 Convergence Speed

- Actor-Critic training time: 13.7 minutes
- DQN training time: 6.3 minutes
- DQN converges around Episode 400; AC around Episode 800

4.3 Loss Curves

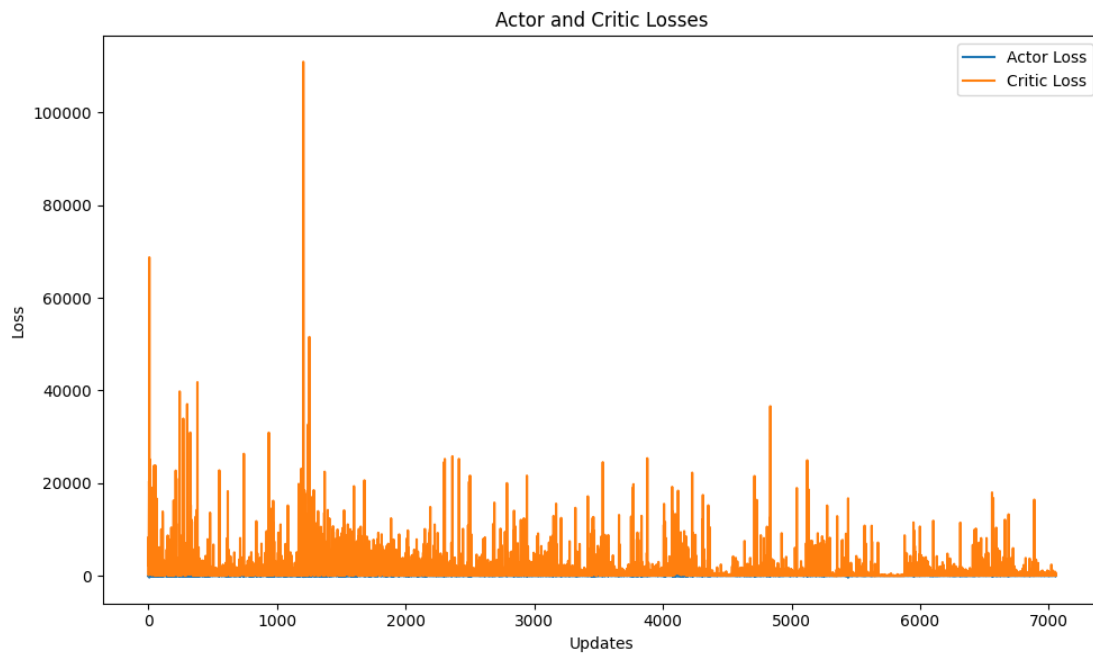


Figure 14: Actor and Critic losses during training.

4.4 Reward Curves

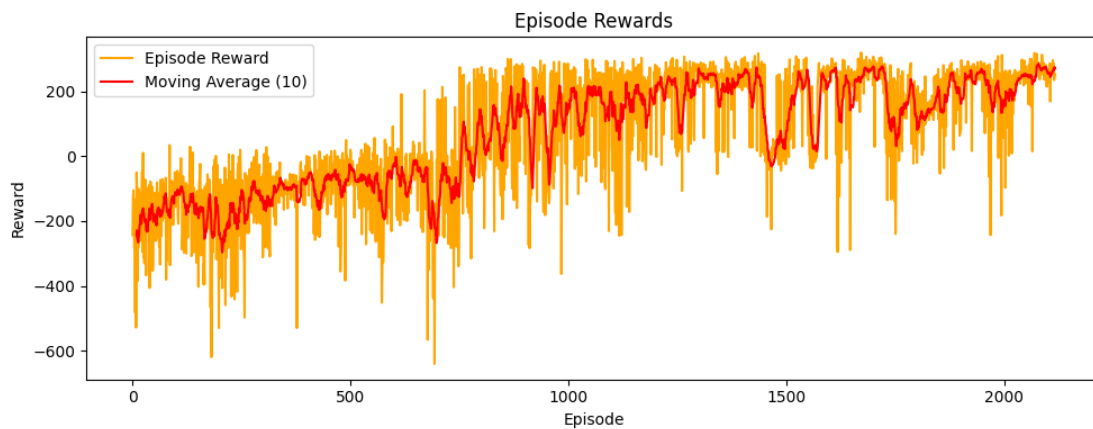


Figure 15: Smoothed Episode Reward (Window=10): Actor-Critic

4.5 Final Evaluation

Algorithm	Mean Reward	Std Dev
Actor-Critic	250.80595	18.79
DQN	274.7656	38.8676

Table 7: Evaluation metrics on 5 episodes.

Supplementary Material

Evaluation GIFs/Videos: https://drive.google.com/drive/folders/1N20N-0PFTVm0ALtmyFrJRu6408c86Mgi?usp=drive_link